

DOCUMENTO DE MODELO ENTIDAD-RELACIÓN (ER)

1. Portada / Encabezado

Nombre del Proyecto:	Synapse Platform
Responsables:	Jhon Lenis, Andrés García, Mariana Bastidas
Fecha:	29 de agosto de 2026
Versión del Documento:	2.0

2. Propósito del Documento

Este documento describe el modelo conceptual de la base de datos de la plataforma **Synapse Platform**, representado mediante un diagrama Entidad-Relación (ER), detallando las entidades principales, atributos y relaciones.

Su objetivo es servir de base para la posterior implementación lógica y física del esquema relacional en PostgreSQL utilizando pgvector y Prisma ORM. Este modelo deriva directamente de los *Casos de Uso (v2.0)* y las *Historias de Usuario (v2.0)*, y respeta los límites definidos en el *Documento de Alcance* y la *Arquitectura del Software*.

3. Alcance del Modelo

El modelo **V2.0** incluye la totalidad de las entidades para los siguientes dominios funcionales:

- **Autenticación, Seguridad y Autorización:** User, Account, Session, Role, Permission, RolePermission, VerificationCode, PasswordReset.
- **Gestión de Contenido (CMS) y Recursos:** Content, Category, Tag, ContentTag, Resource, ContentResource.
- **Eventos y Asistencias:** Event, EventAttendee.
- **Interacciones y Social:** Comment, Reaction, Bookmark, UserPreference.
- **Sistema, Auditoría y Notificaciones:** Notification, AuditLog, SystemSetting, FeatureFlag.

4. Resumen de Entidades

Dominio	Entidad	Descripción
Autenticación y Usuarios	User	Entidad central de cuentas de usuario.
	Account	Cuentas vinculadas a proveedores externos de autenticación (OAuth/NextAuth).
	Session	Almacenamiento y control de sesiones activas.
	VerificationCode	Códigos temporales para validación de correo o inicio de sesión OTP.
	PasswordReset	Control de solicitudes de restablecimiento de contraseña.
Control de Accesos (RBAC)	Role	Roles asignables a los usuarios.
	Permission	Permisos atómicos del sistema.
	RolePermission	Tabla intermedia para la relación N:M entre Roles y Permisos.
Gestión de Contenido (CMS)	Content	Publicaciones, artículos, noticias y entradas generales.

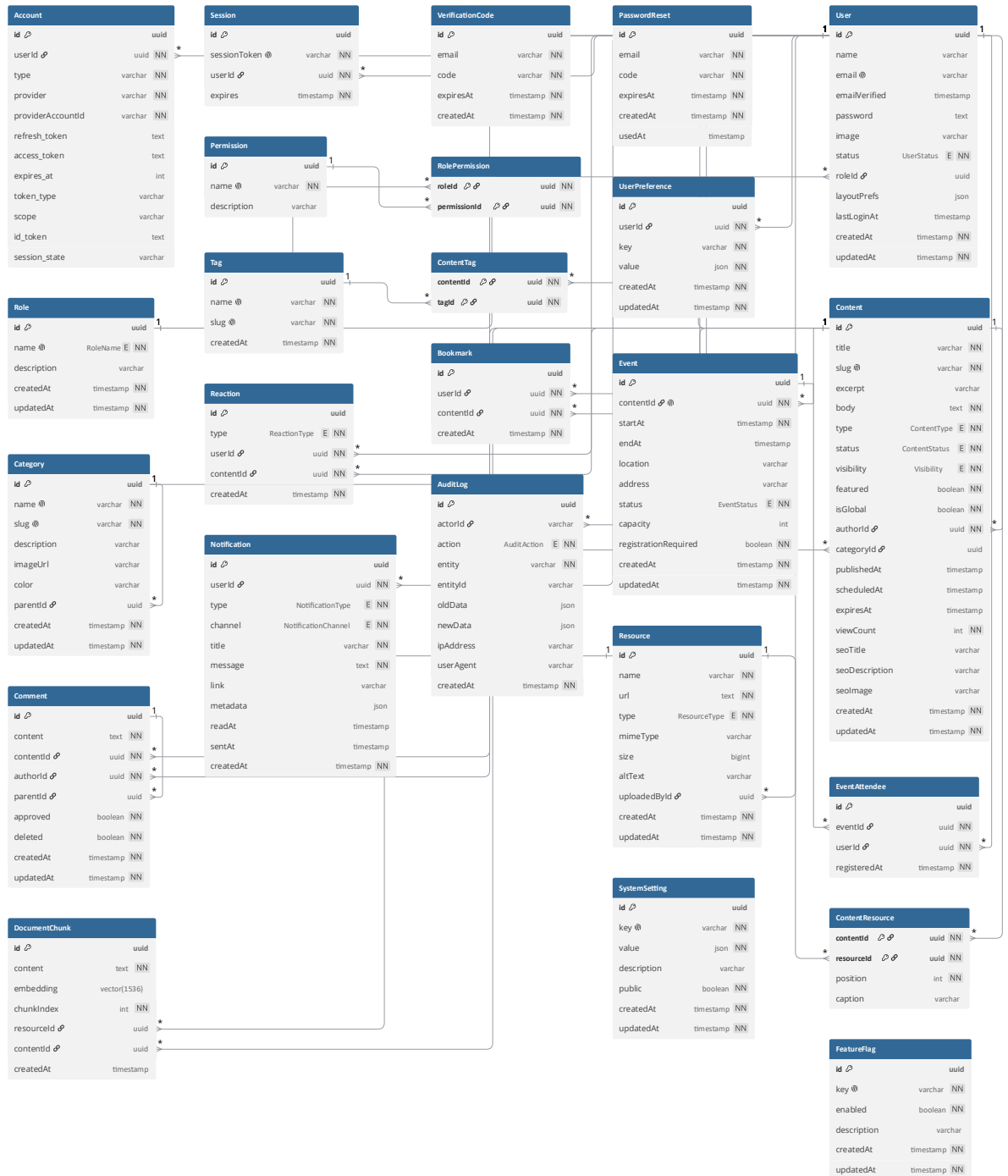
	Category	Estructura jerárquica de categorización (admite parentId).
	Tag	Etiquetas de clasificación libre.
	ContentTag	Relación N:M entre Contenidos y Tags.
	Resource	Archivos multimedia y recursos adjuntos.
	ContentResource	Relación N:M que conecta Recursos con Contenidos (con orden y descripción).
Módulo de Eventos	Event	Extensión de datos para contenidos de tipo evento (fechas, ubicación, aforo).
	EventAttendee	Registro de inscripciones de usuarios a eventos.
Social e Interacción	Comment	Comentarios jerárquicos (soporta respuestas mediante parentId).
	Reaction	Reacciones de usuarios a contenidos (Likes, Emociones, etc.).
	Bookmark	Guardados/Favoritos marcados por los usuarios.

	UserPreference	Configuraciones personalizadas de usuario (clave/valor JSON).
Sistema y Trazabilidad	Notification	Historial de notificaciones internas o multicanal para el usuario.
	AuditLog	Registro estricto de auditoría para trazabilidad de acciones en el sistema.
	SystemSetting	Variables de configuración globales dinámicas en base de datos.

FeatureFlag

Banderas de características para despliegue selectivo de funcionalidades.

5. Diagrama Entidad-Relación (Modelo Conceptual / Lógico)



6. Descripción de Entidades y Atributos

Módulo: Autenticación & Autorización

User

- id (PK, UUID): Identificador único de la cuenta.
- name (VarChar): Nombre del usuario.
- email (VarChar, Unique): Correo electrónico principal.
- emailVerified (Timestamp): Fecha de verificación del correo.
- password (Text, Nullable): Hash de la contraseña.
- image (VarChar, Nullable): URL de la foto de perfil.
- status (Enum: UserStatus): Estado operacional (ACTIVE, INACTIVE, SUSPENDED).
- roleId (FK -> Role.id, UUID): Rol asignado.
- typicalPrefs (JSON, Nullable): Metadatos rápidos de preferencia.
- lastLoginAt (Timestamp, Nullable): Último acceso.
- createdAt / updatedAt (Timestamp): Marcas de tiempo.
- Role, Permission & RolePermission
- Role: id (PK, UUID), name (Enum/VarChar), description (VarChar), createdAt, updatedAt.
- Permission: id (PK, UUID), name (VarChar, Unique), description (VarChar).
- RolePermission: roleId (PK/FK -> Role.id), permissionId (PK/FK -> Permission.id).
- VerificationCode & PasswordReset
- VerificationCode: id (PK, UUID), email (VarChar), code (VarChar), expiresAt (Timestamp), createdAt (Timestamp), usedAt (Timestamp, Nullable).
- PasswordReset: id (PK, UUID), email (VarChar), code (VarChar), expiresAt (Timestamp), createdAt (Timestamp), usedAt (Timestamp, Nullable).

Módulo: Contenido & CMS

Content

- id (PK, UUID): Identificador principal.
- title (VarChar): Título de la entrada.
- slug (VarChar, Unique): Identificador para URLs amigables.
- excerpt (VarChar, Nullable): Resumen breve.
- body (Text): Cuerpo principal del contenido en HTML/Markdown.
- type (Enum: ContentType): Tipo (ARTICLE, NEWS, EVENT, RESOURCE).
- status (Enum: ContentStatus): Estado de publicación (DRAFT, PUBLISHED, ARCHIVED).
- visibility (Enum: Visibility): Accesibilidad (PUBLIC, PRIVATE, RESTRICTED).
- featured / isSticky (Boolean): Opciones de destacado e inmovilizado.
- authorId (FK -> User.id, UUID): Usuario creador.
- categoryId (FK -> Category.id, UUID): Categoría principal.
- publishedAt / scheduledAt / expiresAt (Timestamp): Gestión temporal de publicación.
- viewCount (Int): Contador de visualizaciones.
- seoTitle / seoDescription / seoImage (VarChar): Campos SEO.
- createdAt / updatedAt (Timestamp).

Category & Tag

- Category: id (PK, UUID), name (VarChar), slug (VarChar), description (VarChar), imageUrl (VarChar), color (VarChar), parentId (FK -> Category.id, Self-reference para subcategorías), createdAt, updatedAt.
- Tag: id (PK, UUID), name (VarChar), slug (VarChar), createdAt, updatedAt.
- ContentTag: contentId (PK/FK -> Content.id), tagId (PK/FK -> Tag.id).

Resource, ContentResource & DocumentChunk (*Motor RAG*)

- **Resource:** id (PK, UUID), name (VarChar), url (Text), type (Enum: ResourceType), mimeType (VarChar), size (BigInt), altText (VarChar), uploadedById (FK -> User.id), createdAt, updatedAt.
- **ContentResource:** contentId (PK/FK -> Content.id), resourceId (PK/FK -> Resource.id), position (Int), caption (VarChar).
- **DocumentChunk** (*Fragmentos vectoriales para Inteligencia Artificial*):
 - id (PK, UUID): Identificador del chunk.
 - content (Text): Texto del fragmento extraído.
 - embedding (Unsupported / Vector): Embeddings vectoriales de alta dimensión (pgvector).
 - chunkIndex (Int): Orden secuencial en el documento original.
 - resourceId (FK -> Resource.id, Optional): Recurso de origen si proviene de un archivo cargado.
 - contentId (FK -> Content.id, Optional): Publicación de origen si proviene del CMS.
 - createdAt (Timestamp)



Módulo: Eventos

Event

- id (PK, UUID): Identificador del evento.
- contentId (FK -> Content.id, UUID): Referencia al contenido asociado.
- startAt / endAt (Timestamp): Fechas y horas de inicio/fin.
- location (VarChar): Nombre del lugar/plataforma.
- address (VarChar, Nullable): Dirección física.
- status (Enum: EventStatus): Estado (SCHEDULED, CANCELLED, COMPLETED).
- capacity (Int, Nullable): Límite de aforo.
- registrationRequired (Boolean): Requiere registro previo.

- createdAt / updatedAt (Timestamp).

EventAttendee

- id (PK, UUID): Identificador de la inscripción.
- eventId (FK -> Event.id, UUID).
- userId (FK -> User.id, UUID).
- registeredAt (Timestamp).

Módulo: Social e Interacciones

- Comment: id (PK, UUID), content (Text), authorId (FK -> User.id), contentId (FK -> Content.id), parentId (FK -> Comment.id, Autorreferencial para hilos/respuestas), approved (Boolean), deletedAt (Timestamp, Nullable), createdAt, updatedAt.
- Reaction: id (PK, UUID), type (Enum: ReactionType), userId (FK -> User.id), contentId (FK -> Content.id), createdAt.
- Bookmark: id (PK, UUID), userId (FK -> User.id), contentId (FK -> Content.id), createdAt.
- UserPreference: id (PK, UUID), userId (FK -> User.id), key (VarChar), value (JSON), createdAt, updatedAt.

Módulo: Sistema, Notificaciones & Auditoría

- Notification: id (PK, UUID), userId (FK -> User.id), type (Enum: NotificationType), channel (Enum: NotificationChannel), title (VarChar), message (Text), link (VarChar), metadata (JSON), readAt (Timestamp, Nullable), sentAt (Timestamp, Nullable), createdAt.
- AuditLog: id (PK, UUID), actorId (VarChar / FK -> User.id), action (Enum: AuditAction), entity (VarChar), entityId (VarChar), oldData (JSON), newData (JSON), ipAddress (VarChar), userAgent (VarChar), createdAt.
- SystemSetting: id (PK, UUID), key (VarChar, Unique), value (JSON), description (VarChar), public (Boolean), createdAt, updatedAt.

- FeatureFlag: id (PK, UUID), key (VarChar, Unique), enabled (Boolean), description (VarChar), createdAt, updatedAt.

7. Descripción de Relaciones

Entidad Orig.	Entidad Dest.	Relación	Cardinalidad	Descripción
Role	User	Asigna	1:N	Un rol pertenece a múltiples usuarios.
Role	Permission	Define	N:M	Vía tabla intermedia RolePermission.
User	Content	Autora	1:N	Un usuario crea múltiples contenidos.
Category	Category	Jerarquiza	1:N	Autorreferencial (parentId) para subcategorías.
Category	Content	Clasifica	1:N	Una categoría agrupa contenidos.
Content	Tag	Etiqueta	N:M	Vía tabla intermedia ContentTag.

Content	Resource	Adjunta	N:M	Vía tabla intermedia ContentResource.
Content	Event	Extiende	1:1	Un contenido de tipo EVENT posee sus metadatos en Event.
Event	User	Inscribe	N:M	Vía tabla intermedia EventAttendee.
Content	Comment	Recibe	1:N	Un contenido recibe múltiples comentarios.
Comment	Comment	Responde	1:N	Autorreferencial (parentId) para hilos de comentarios.
User	Notification	Recibe	1:N	Notificaciones dirigidas al usuario.
User	AuditLog	Ejecuta	1:N	Trazabilidad de acciones efectuadas por el usuario.

8. Reglas de Negocio Relacionadas a la Base de Datos

- a. **Unicidad de Slugs:** Los campos slug en Content, Category y Tag deben ser únicos para garantizar el enrutamiento SEO.
- b. **Jerarquías Seguras:** Las autorreferencias en Category y Comment deben contar con validaciones a nivel de backend u ORM para prevenir ciclos infinitos (ej. una categoría no puede ser madre de sí misma).
- c. **Control de Aforo en Eventos:** La inserción en EventAttendee debe verificar que la cantidad de registros no supere la capacity definida en la entidad Event (si capacity no es NULL).
- d. **Soft Delete / Borrado Lógico en Comentarios:** La entidad Comment implementa borrado lógico mediante la columna deletedAt para preservar el hilo de la conversación.
- e. **Inmutabilidad en AuditLog:** La tabla AuditLog únicamente permite operaciones INSERT y SELECT. Quedan prohibidos los UPDATE y DELETE.

9. Normalización Aplicada

El modelo cumple rigurosamente con la **Tercera Forma Normal (3FN)**:

- **1FN:** Todos los atributos contienen valores atómicos (uso de tablas intermedias como ContentTag y RolePermission para relaciones multivaluadas).
- **2FN:** Todos los atributos no clave dependen en su totalidad de la clave primaria de sus respectivas tablas.
- **3FN:** No existen dependencias transitivas. Los datos de auditoría, preferencias y detalles de roles están desacoplados en sus propias entidades.

10. Anexos

ORM – PRISMA SCHEMA

```
datasource db {  
  provider = "postgresql"  
  url      = env("DATABASE_URL")  
}
```

```
}

generator client {
  provider = "prisma-client-js"
}

// =====
// ENUMS
// =====

enum UserStatus {
  ACTIVE
  INACTIVE
  SUSPENDED
}

enum ContentType {
  ARTICLE
  NEWS
  EVENT
  RESOURCE
}

enum ContentStatus {
  DRAFT
  PUBLISHED
  ARCHIVED
}

enum Visibility {
  PUBLIC
  PRIVATE
  RESTRICTED
}

enum ResourceType {
  IMAGE
  DOCUMENT
}
```

```
VIDEO
AUDIO
OTHER
}
```

```
enum EventStatus {
    SCHEDULED
    CANCELLED
    COMPLETED
}
```

```
enum ReactionType {
    LIKE
    LOVE
    CELEBRATE
    INSIGHTFUL
}
```

```
enum NotificationType {
    INFO
    SUCCESS
    WARNING
    SYSTEM
}
```

```
enum NotificationChannel {
    IN_APP
    EMAIL
    PUSH
}
```

```
enum AuditAction {
    CREATE
    UPDATE
    DELETE
    LOGIN
    LOGOUT
}
```

```

// =====
// AUTENTICACIÓN Y ROLES
// =====

model User {
  id      String  @id @default(uuid())
  name    String
  email   String  @unique
  emailVerified DateTime?
  password String?
  image   String?
  status  UserStatus @default(ACTIVE)
  roleId  String
  role    Role      @relation(fields: [roleId], references: [id])
  typicalPrefs Json?
  lastLoginAt DateTime?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  // Relaciones
  accounts Account[]
  sessions Session[]
  contents Content[]
  resources Resource[]
  eventAttendees EventAttendee[]
  comments Comment[]
  reactions Reaction[]
  bookmarks Bookmark[]
  userPrefs UserPreference[]
  notifications Notification[]
  auditLogs AuditLog[]
}

model Role {
  id      String  @id @default(uuid())
  name    String  @unique
  description String?

```

```

    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt

    users User[]
    permissions RolePermission[]
}

model Permission {
  id String @id @default(uuid())
  name String @unique
  description String?

  roles RolePermission[]
}

model RolePermission {
  roleId String
  permissionId String
  role Role @relation(fields: [roleId], references: [id], onDelete: Cascade)
  permission Permission @relation(fields: [permissionId], references: [id],
onDelete: Cascade)

  @@id([roleId, permissionId])
}

model Account {
  id String @id @default(uuid())
  userId String
  type String
  provider String
  providerAccountId String
  refreshToken String? @db.Text
  accessToken String? @db.Text
  expiresAt Int?
  tokenType String?
  scope String?
  idToken String? @db.Text
  sessionState String?

```

```
    user User @relation(fields: [userId], references: [id], onDelete: Cascade)

    @@unique([provider, providerAccountId])
}
```

```
model Session {
  id      String @id @default(uuid())
  sessionToken String @unique
  userId  String
  expires DateTime
}
```

```
    user User @relation(fields: [userId], references: [id], onDelete: Cascade)
}
```

```
model VerificationCode {
  id      String @id @default(uuid())
  email   String
  code    String
  expiresAt DateTime
  createdAt DateTime @default(now())
  usedAt  DateTime?
}
```

```
model PasswordReset {
  id      String @id @default(uuid())
  email   String
  code    String
  expiresAt DateTime
  createdAt DateTime @default(now())
  usedAt  DateTime?
}
```

```
// =====
// CMS Y GESTIÓN DE CONTENIDOS
// =====
```

```
model Content {
```

```

id      String   @id @default(uuid())
title   String
slug    String   @unique
excerpt String?
body    String   @db.Text
type    ContentType
status  ContentStatus @default(DRAFT)
visibility Visibility @default(PUBLIC)
featured Boolean   @default(false)
isSticky Boolean    @default(false)
authorId String
author  User        @relation(fields: [authorId], references: [id])
categoryId String
category Category    @relation(fields: [categoryId], references: [id])
publishedAt DateTime?
scheduledAt DateTime?
expiresAt  DateTime?
viewCount  Int        @default(0)
seoTitle   String?
seoDescription String?
seoImage   String?
createdAt  DateTime    @default(now())
updatedAt  DateTime    @updatedAt

// Relaciones
tags      ContentTag[]
resources ContentResource[]
events    Event[]
comments  Comment[]
reactions Reaction[]
bookmarks Bookmark[]
chunks    DocumentChunk[]
}

```

```

model Category {
  id      String @id @default(uuid())
  name    String
  slug    String @unique

```

```

description String?
imageUrl String?
color String?
parentId String?
parent Category? @relation("SubCategories", fields: [parentId], references:
[id])
children Category[] @relation("SubCategories")
createdAt DateTime @default(now())
updatedAt DateTime @updatedAt

contents Content[]
}

```

```

model Tag {
  id String @id @default(uuid())
  name String @unique
  slug String @unique
  createdAt DateTime @default(now())

  contents ContentTag[]
}

```

```

model ContentTag {
  contentId String
  tagId String
  content Content @relation(fields: [contentId], references: [id], onDelete:
Cascade)
  tag Tag @relation(fields: [tagId], references: [id], onDelete: Cascade)

  @@id([contentId, tagId])
}

```

```

model Resource {
  id String @id @default(uuid())
  name String
  url String @db.Text
  type ResourceType
  mimeType String
}

```

```

size      BigInt
altText   String?
uploadedById String
uploadedBy User    @relation(fields: [uploadedById], references: [id])
createdAt DateTime @default(now())
updatedAt  DateTime @updatedAt

contents ContentResource[]
chunks DocumentChunk[]
}

model ContentResource {
  contentId String
  resourceId String
  position Int?
  caption String?
  content Content @relation(fields: [contentId], references: [id], onDelete: Cascade)
  resource Resource @relation(fields: [resourceId], references: [id], onDelete: Cascade)

  @@id([contentId, resourceId])
}

// ===== // MOTOR RAG E
// INTELIGENCIA ARTIFICIAL //
// ===== model DocumentChunk {
id String      @id      @default(uuid())
content String  @db.Text
// Representación del campo vectorial para PostgreSQL / pgvector
embedding Unsupported("vector(1536)")?
chunkIndex Int

// Relaciones opcionales (puede provenir de un archivo adjunto o de un post)
resourceId String?
resource Resource? @relation(fields: [resourceId], references: [id], onDelete: Cascade)

```

```
contentId String? contentRef Content? @relation(fields: [contentId],
references: [id], onDelete: Cascade)
```

```
createdAt DateTime @default(now())
```

```
@@index([resourceId])
```

```
@@index([contentId])
```

```
}
```

```
// =====
```

```
// MÓDULO DE EVENTOS
```

```
// =====
```

```
model Event {
```

```
  id          String  @id @default(uuid())
```

```
  contentId   String
```

```
  content     Content  @relation(fields: [contentId], references: [id],
onDelete: Cascade)
```

```
  startAt     DateTime
```

```
  endAt       DateTime
```

```
  location    String
```

```
  address     String?
```

```
  status      EventStatus @default(SCHEDULED)
```

```
  capacity    Int?
```

```
  registrationRequired Boolean @default(false)
```

```
  createdAt   DateTime @default(now())
```

```
  updatedAt   DateTime @updatedAt
```

```
  attendees EventAttendee[]
```

```
}
```

```
model EventAttendee {
```

```
  id      String  @id @default(uuid())
```

```
  eventId String
```

```
  userId  String
```

```
  registeredAt DateTime @default(now())
```

```
  event Event @relation(fields: [eventId], references: [id], onDelete: Cascade)
```

```

user User @relation(fields: [userId], references: [id], onDelete: Cascade)

@@unique([eventId, userId])
}

// =====
// SOCIAL E INTERACCIONES
// =====

model Comment {
  id String @id @default(uuid())
  content String @db.Text
  authorId String
  contentId String
  parentId String?
  approved Boolean @default(true)
  deletedAt DateTime?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  author User @relation(fields: [authorId], references: [id])
  post Content @relation(fields: [contentId], references: [id], onDelete: Cascade)
  parent Comment? @relation("CommentReplies", fields: [parentId], references: [id])
  replies Comment[] @relation("CommentReplies")
}

model Reaction {
  id String @id @default(uuid())
  type ReactionType
  userId String
  contentId String
  createdAt DateTime @default(now())

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
  content Content @relation(fields: [contentId], references: [id], onDelete: Cascade)
}

```

```
    @@unique([userId, contentId, type])
}
```

```
model Bookmark {
  id    String @id @default(uuid())
  userId String
  contentId String
  createdAt DateTime @default(now())
```

```
  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
  content Content @relation(fields: [contentId], references: [id], onDelete: Cascade)
```

```
    @@unique([userId, contentId])
}
```

```
model UserPreference {
  id    String @id @default(uuid())
  userId String
  key    String
  value  Json
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
```

```
  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
```

```
    @@unique([userId, key])
}
```

```
// =====
// AUDITORÍA Y SISTEMA
// =====
```

```
model Notification {
  id    String @id @default(uuid())
  userId String
  type  NotificationType
```

```

channel NotificationChannel @default(IN_APP)
title String
message String @db.Text
link String?
metadata Json?
readAt DateTime?
sentAt DateTime?
createdAt DateTime @default(now())

user User @relation(fields: [userId], references: [id], onDelete: Cascade)
}

```

```

model AuditLog {
  id String @id @default(uuid())
  actorId String
  action AuditAction
  entity String
  entityId String
  oldData Json?
  newData Json?
  ipAddress String?
  userAgent String?
  createdAt DateTime @default(now())

  actor User @relation(fields: [actorId], references: [id])
}

```

```

model SystemSetting {
  id String @id @default(uuid())
  key String @unique
  value Json
  description String?
  public Boolean @default(false)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}

```

```

model FeatureFlag {

```

```
id      String @id @default(uuid())
key     String @unique
enabled Boolean @default(false)
description String?
createdAt DateTime @default(now())
updatedAt DateTime @updatedAt
}
```

11. Historial de Cambios

Versión	Fecha	Descripción del Cambio	Responsable
2.0	29/08/2026	Reestructuración a la Versión 2.0. Integración de entidades para OtpToken (login sin contraseña), DocumentChunk con soporte pgvector para el motor RAG, y eliminación de tablas no aplicables al MVP.	Equipo de Desarrollo